



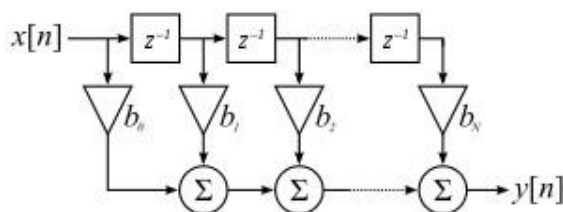
In this Lab 7 you are to implement a finite impulse response (FIR) digital filter and an infinite impulse response (IIR) in Xilinx Vivado and SDK, standalone OS (not FreeRTOS) and the Zynq Zybo Board with the PmodAD1 ADC and PmodDA2 DAC external peripherals. Labs 5 and 6 can form the basis for Lab 7. The PmodAD1 ADC and PmodDA2 DAC program now *AD1DA2Pmod.c* is provided

as a template on Canvas and shown below.

PART 1 (straight through ADC-to-DAC with a noisy triangular waveform):

- 1) Create a Vivado Project that includes both the PmodAD1 ADC and the PmodDA2 DAC.
- 2) Using two channels of the Discovery 2 waveform generator, Generate a triangular waveform (1KHz, 1V Amplitude, 1V Offset), and a Noise Source (500KHz, 500mV Amplitude, and 500mV Offset).
- 3) Input the waveforms to the two ADC input channels.
- 4) In SDK, generate a noisy triangular waveform by adding the Noise source to the triangular waveform.
- 5) In SDK, send the noisy triangular waveform straight through to the DAC and use two channels of the Discovery 2 Oscilloscope to compare the clean triangular waveform to the noisy triangular waveform.

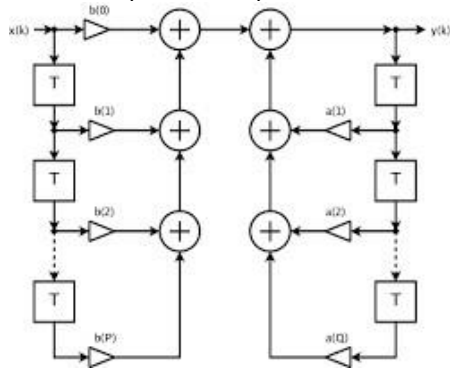
PART 2 (FIR Filter):



- 1) In SDK create an FIR Filter with the following parameters:
 - 2 Zeros @ $(-0.7, \pm 0.1j)$
 - 2 Poles Both @ $(0,0)$

- 2) Filter the noisy triangular waveform, and use two channels of the Discovery 2 Oscilloscope to compare the noisy triangular waveform to the FIR filtered triangular waveform.

PART 3 (IIR Filter):



- 1) In SDK create an IIR Filter with the following parameters:
 2 Zeros @ $(-0.7, \pm 0.1j)$
 2 Poles @ $(0.7, \pm 0.1j)$
- 2) Filter the noisy triangular waveform, and use two channels of the Discovery 2 Oscilloscope to compare the noisy triangular waveform to the IIR filtered triangular waveform.

In your report, compare the performance of the FIR and IIR filter. Which one worked better? Why?

TIPS:

- The code below shows an example of an FIR filter on ADC/DAC channel 1 and an example of a straight through configuration on ADC/DAC channel 2.
- As shown in the template program *AD1DA2Pmod.c* below, the ADC is unipolar and must be:
 - Offset by -2048 prior to filtering; and
 - Scaled prior to filtering, by an integer of at least the sum of 2048 times each of the filter coefficients divided by the ADC range of 4096. For example, in the code below, the scale factor is 8 because $(2048(1+3+4+3+5))/4096 = 8$. Your scaling factor may differ. If you

are seeing signal clipping of the filtered signal on the oscilloscope, your scale factor is too small. This determination should be included in the Laboratory Report.

This is a two-week Lab

```
//AD1DA2Pmod.c PmodAD1 PmodDA2 ADC- IIR/FIR Digital Filter Example
```

```
#include "xparameters.h"
#include "xil_io.h"
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "sleep.h"
#include "math.h"
```

```
//AD1Pmod from Address Editor in Vivado, first IP
```

```
#define AD1acq      0x43C00000 //AD1 acquisition - output
#define AD1dav      0x43C00004 //AD1 data available - input
#define AD1dat1     0x43C00008 //AD1 channel 1 data - input
#define AD1dat2     0x43C0000C //AD1 channel 2 data - input
```

```
//DAC2Pmod from Address Editor in Vivado, second IP
```

```
#define DA2acq      0x43C10000 //DA2 acquisition - output
#define DA2dav      0x43C10004 //DA2 data available - input
#define DA2dat1     0x43C10008 //DA2 channel 1 data - output
#define DA2dat2     0x43C1000C //DA2 channel 2 data - output
```

```
int main(void)
```

```
{
    int adcdav; //ADC data available int
    adcdat1; //ADC channel 1 data int
    adcdat2; //ADC channel 2 data
```

```
    int dacdata1; //DAC channel 1 data
    int dacdata2; //DAC channel 2 data int
    dacdav; //DAC data available
```

```
    int x1a; //FIR x(m)
    int x1b; // x(m-1)
    int x1c; // x(m-2) int
    x1d; // x(m-3) int x1e;
    // x(m-4)
    int y1a; // y(m)
```

```

    xil_printf("\n\rStarting AD1-DA2 Pmod FIR example...\n");
    Xil_Out32(AD1acq,0); //ADC stop acquire adcdav=Xil_In32(AD1dav);
    //ADC available?
    while(adcdav==1)
    adcdav=Xil_In32(AD1dav);      Xil_Out32(DA2acq,0);
    //DAC stop acquire dacdav=Xil_In32(DA2dav); //DAC
    available?
        while(dacdav==1)
    dacdav=Xil_In32(DA2dav); while (1)
    {
        //ADC
        Xil_Out32(AD1acq,1); //ADC acquire
    while (adcdav==0) //ADC data available?
    adcdav=Xil_In32(AD1dav);
        Xil_Out32(AD1acq,0); //ADC stop acquire
        adcdat1=Xil_In32(AD1dat1); //input ADC data
    adcdat2=Xil_In32(AD1dat2);
    while (adcdav==1) //wait for reset
    adcdav=Xil_In32(AD1dav);

    //DAC
        Xil_Out32(DA2dat1, dacdat1); //output DAC data
        Xil_Out32(DA2dat2, dacdat2);
        Xil_Out32(DA2acq,1); //DAC acquire
    while (dacdav==0) //DAC data output?
    dacdav=Xil_In32(DA2dav);
        Xil_Out32(DA2acq,0); //stop DAC acquire
    while(dacdav==1) //wait for reset
    dacdav=Xil_In32(DA2dav);
    }
}

```